Code-Layout, Readability and Reusability

Date	10 july 2026
Team ID	
Project Name	StudyAI-Smart AI educate Companion&Quiz Generator
Maximum Marks	5 Marks

Code-Layout, Readability and Reusability:

This document evaluates the quality of the codebase in terms of its structure, readability, and reusability. Well-organized code with proper naming conventions, comments, and modular design ensures that the project is maintainable and can be extended or reused in future development.

Code Layout Checklist:

S.No.	Code Quality Parameter	Description	Followed (Yes / No / Partial)	Remarks
1	Consistent Indentation	Uniform spacing/tabs used throughoutthe code	Yes	Enforced via Prettier/ESLint on frontend, PEP8 on backend
2	Proper File Structure	Files and folders are logically organized	Yes	Separate components/, pages/, routes/, utils/ folders
3	Meaningful Variable Names	Variables reflect their purpose clearly	Yes	e.g., uploadedFile, quizScore, summaryText
4	Function / Method Names	Functions are descriptively named	Yes	e.g., generateFlashcards(), parseDocument(), getUserAnalytics()
5	Code Comments	Inline and block comments explain logc	Partial	Core AI prompt logic commented; minor UI components need more comments
6	Modular Design	Code is split into reusable functions/mdules	Yes	Reusable React components (FlashcardCard, QuizQuestion, Dashboard)
7	No Redundant Code	Duplicate or unused code is removed	Yes	Shared API call logic centralized in an Axios instance
8	Error Handling	Exceptions and errors are handled gracefully	Yes	Try-catch around Groq API calls and file parsing with user-friendly error messages

Reusable Components / Modules:

S.No	Component / Module Name	Language / Technology	Where Reused	Reusability Level (High / Medium / Low)
1	FlashcardCard	React (JSX)	Flashcard view, Quiz review screen	High

2	API client (Axios instance)	JavaScript	All frontend API calls (auth, upload, quiz, analytics)	High
3	Auth Middleware (JWT verify)	Python/Flask	All protected backend routes	High
4	Groq Prompt Builder	Python	Summary, flashcard, and quiz generation functions	Medium
5	Chart Component	React + Chart.js	Analytics dashboard, quiz history view	Medium

Overall Code Quality Assessment:

Aspect	**Rating (1-5)**	**Comments**
Code Layout & Structure	4	Clear folder separation between frontend/backend
Readability	4	Consistent naming; some functions could use more comments
Reusability	4	Shared components reduce duplication effectively
Documentation / Comments	3	README present; inline comments can be improved
Overall Score	15/20	Solid, maintainable codebase overall